Filesmodes + Unicode

1521 tutorial 9

01

Environment Variables

What is an environment variable?

- When run, a program is passed a set of environment variables
- this is an array of strings of the form *'name=value'* terminated with NULL
- this is accessed through the global variable environ

Access them using:

getenv()

setenv()

02

File Permissions

The Stat Struct

- The stat() and lstat() functions both take an argument which is a pointer to a struct stat object, and fill it with the meta-data for a named file.

int stat(const char *pathname, struct stat *statbuf)

pathname of file

An empty struct stat to put the information in

The Stat Struct

```
struct stat {
  dev_t     st_dev;         /* ID of device containing file */
  ino_t     st_ino;         /* Inode number */
  mode_t    st_mode;        /* File type and mode */
  nlink_t   st_nlink;       /* Number of hard links */
  uid_t     st_uid;         /* User ID of owner */
  gid_t     st_gid;         /* Group ID of owner */
  dev_t     st_rdev;        /* Device ID (if special file) */
  off_t     st_size;        /* Total size, in bytes */
  blksize_t st_blksize;     /* Block size for filesystem I/O */
  blkcnt_t  st_blocks;      /* Number of 512B blocks allocated */
  struct timespec st_atim;  /* Time of last access */
  struct timespec st_mtim;  /* Time of last modification */
  struct timespec st_ctim;  /* Time of last status change */
};
```

The Stat Struct

ino_t st_ino

An inode number, giving an index into the filesystem's table of file metadata structures. For stat.c, it could be any largish positive integer. The inode number can be accessed using ls -li.

mode_t st_mode

Contains information about the file type and the file permissions, encoded as a bit-string. These bit-strings are usually written in octal, to make it easy to see the 3 groups of 3 bits defining the file permissions. A regular file like stat.c would have an st_mode value of 0100644 (from S_IFREG | S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH).

uid_t st_uid

Gives the numeric user id (uid) of the user to whom the file belongs (its owner); in this case, 516. This can be retrieved using ls -ln.

gid_t st_gid

Gives the numeric id (gid) of the group to which the file belongs; in this case, 36820. This can be retrieved using ls -ln.

The Stat Struct

off_t st_size

Gives the total size of the file in bytes. For a text file like stat.c, it's simply the number of characters in the file's content (i.e., 1855).

blksize_t st_blksize

Gives the size of a block on the storage device useful for files of this type. Typical block size are 512, 1024, 4096, 8192.

blkcnt_t st_blocks

Gives the amount of space on the storage device allocated for this file. Since it's allocated in 512B chunks, more space might be allocated than is actually required to store the bytes. Often blocks are allocated in groups of size 2^n. The total bytes allocated in the blocks must, of course, be larger than st_size. For stat.c, there are 8 blocks allocated (a total of 4096 bytes, to store the 1855 actually in the file).

The Stat Struct

time_t st_mtime

Gives the last time the file was modified. A time_t value is typically implemented as an integer giving the number of seconds since midnight on Jan 1 1970. For the stat.c file, the most recent update time is shown in the ls output as Sep 9 14:24, which, here, is implied to be 2017/09/09 14:24, a value around 1504931040.

time_t st_atime

The last time the file content was accessed (read or written). This value can be retrieved using ls -lu.

time_t st_ctime

The last time the file status was changed. This could mean changing the file contents, or changing its associated metadata. This value can be retrieved using ls -lc.

Modes

R

read

Read permission is used to access the file's contents. You can use a tool like cat or less on the file to display the file contents.

W

write

Write permission allows you to modify or change the contents of a file.

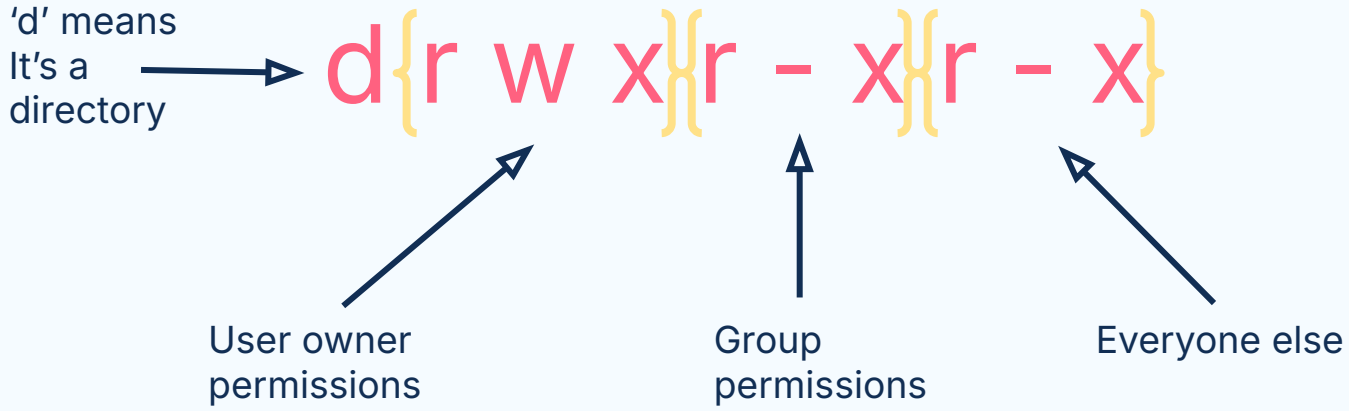
X

execute

Execute permission allows you to execute the contents of a file. Typically, executables would be things like commands or compiled binary applications.

Modes

The string represents which permissions correspond to which user.



Modes

We can also represent each set of 3 in binary!

d{r w x}{r - x}{r - x}

1 1 1 1 01 1 01

7 5 5

Modes

We can also represent each set of 3 in binary!

chmod -r 777 file.exe

777 = all modes permitted for everyone

Modes

- Chmod allows us to change the mode of a file.

```
int chmod(const char *pathname, mode_t mode);
```

Pathname of the file

The new file mode is specified in *mode*, which is a bit mask created by ORing together zero or more flags.

We can also use octal!

Modes

st_mode is a bitwise-or of these values (& others):

```
S_IFLNK    0120000   symbolic link
S_IFREG    0100000   regular file
S_IFBLK    0060000   block device
S_IFDIR    0040000   directory
S_IFCHR    0020000   character device
S_IFIFO    0010000   FIFO
S_IRUSR    0000400   owner has read permission
S_IWUSR    0000200   owner has write permission
S_IXUSR    0000100   owner has execute permission
S_IRGRP    0000040   group has read permission
S_IWGRP    0000020   group has write permission
S_IXGRP    0000010   group has execute permission
S_IROTH    0000004   others have read permission
S_IWOTH    0000002   others have write permission
S_IXOTH    0000001   others have execute permission
```

04
Unicode

What is the point of unicode?

- modern computers use unicode to represent text
- While mathematical expressions are used for numbers such as two’s complement and IEEE 754, character encoding is done through a lookup table.
- Ranges from 0x0000 to 0x10FFFF

What is the point of unicode?

- So how do we store all of these representations?
- Ranges from 0x0000 to 0x10FFFF so we need at least 21 bits to represent them.
- Maybe we could use 32 bits?

What is the point of unicode?

- UTF-32 works but..
- 21 bits for the largest code point - wastes 11 bits
- For ascii characters that are in the 0th plane it wastes 16!

What is the point of unicode?

- Introducing UTF-8!

This is the layout of UTF-8

#bytes	#bits	Byte 1	Byte 2	Byte 3	Byte 4
1	7	0xxxxxxx	-	-	-
2	11	110xxxxx	10xxxxxx	-	-
3	16	1110xxxx	10xxxxxx	10xxxxxx	-
4	21	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx